AI with Rav

Learn AI the way you actually think.

DAY 7 of 30

PYTHON

If / Elif / Else — the road that forks

WHAT YOU'LL LEARN TODAY

- if — do something only when True
- The shape: colon and indent
- else — the other road
- elif — more than two paths
- It stops at the first match

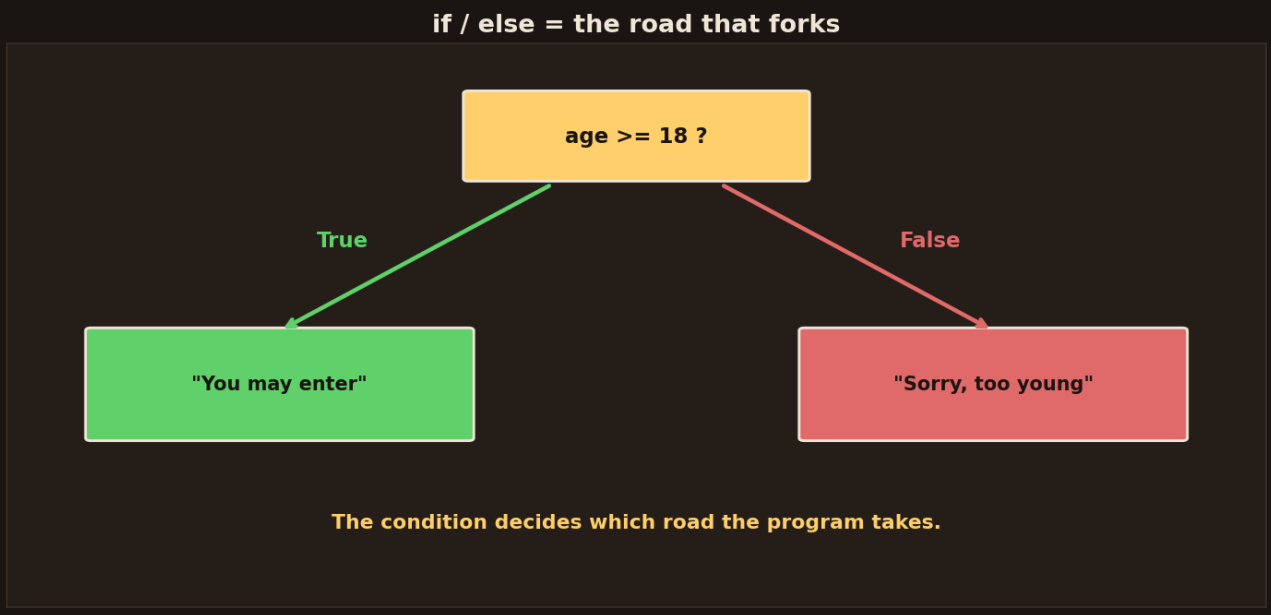
If / Elif / Else — the road that forks

> In One Line: Yesterday you learned to ASK yes/no questions. Today the program ACTS on the answer. `if` runs some code only when a condition is True — like a road that forks. This is where your code stops being a straight line and starts making real decisions.

Start here: a road that forks

Until now your programs ran in a straight line, top to bottom. But real programs need to **choose:** show the dashboard *if* logged in, give a discount *if* it's a member, warn *if* the tank is low. Python does this with `if` — think of it as a **fork in the road.** The condition decides which way the program goes.

if — do something only when True



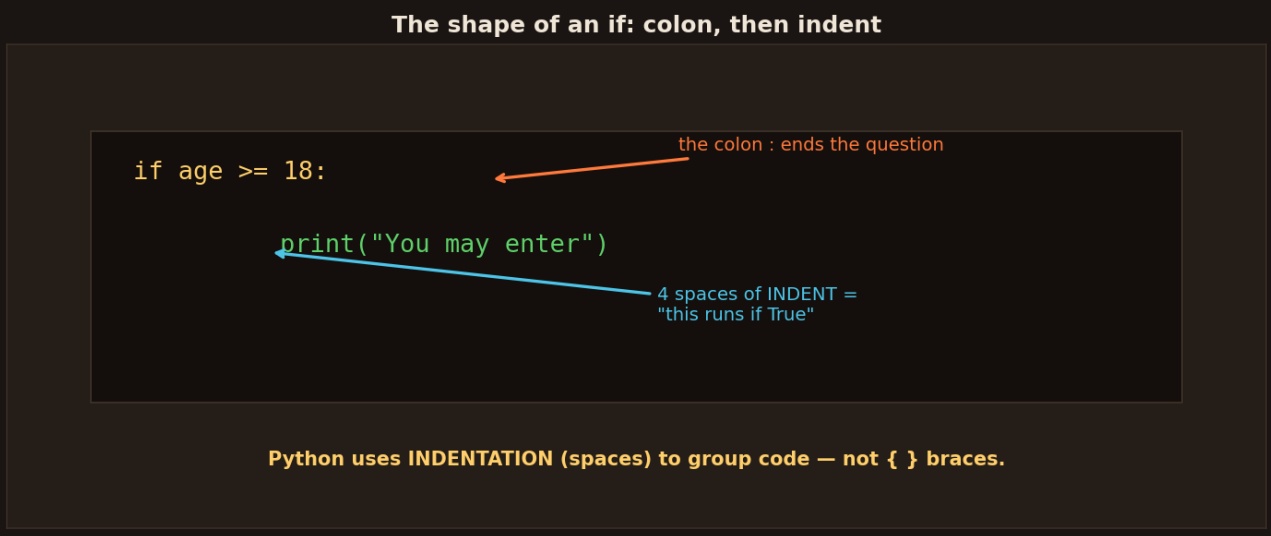
if/else is a road that forks. The question age >= 18 has two roads: True leads to "You may enter" (green), False leads to "Sorry, too young" (red). The condition decides which road the program takes.

The simplest form: "**if** this is true, do that."

```
age = 20
if age >= 18:
    print("You may enter")
# -> You may enter
```

> Read it in English: "if age is 18 or more, print the welcome." If the condition is `True`, the indented line runs. If it's `False`, Python simply skips it and moves on. That's the core of every decision your code will ever make.

The shape of an if — colon and indent



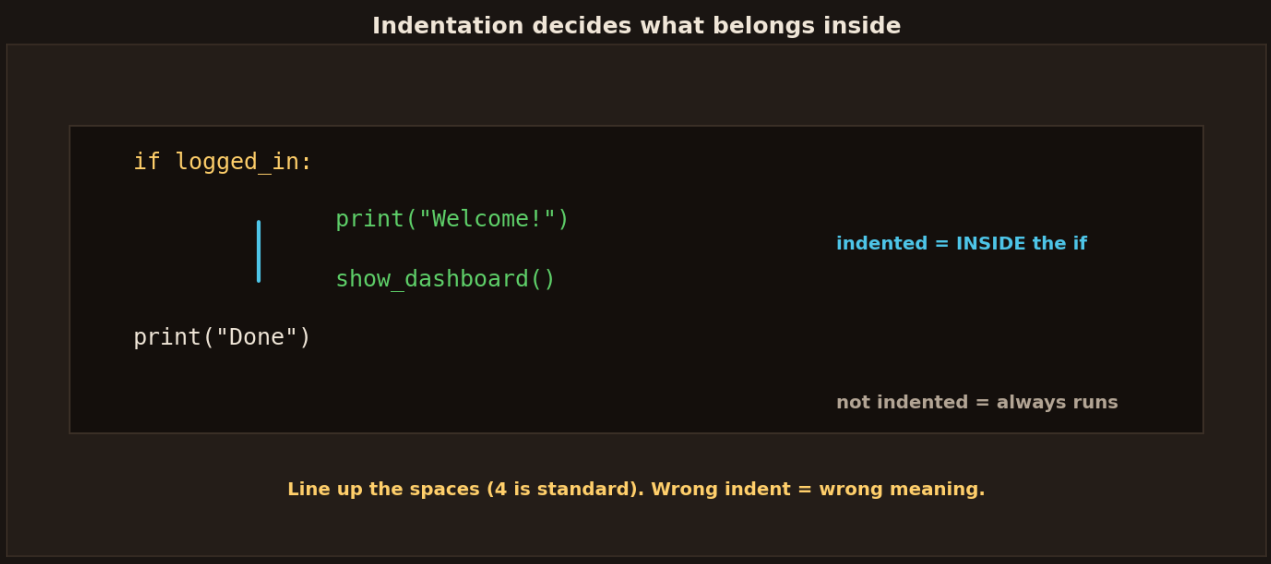
The shape of an if: the line ends with a colon (:), then the next line is indented 4 spaces. The colon ends the question; the indent means "this runs if True". Python uses indentation to group code, not curly braces.

Two things about the *shape* trip up beginners, so let's name them:

- The `if` line **ends with a colon** `:` — that colon says "here comes what to do."
- The next line is **indented** (4 spaces) — the indent means "this belongs inside the if."

> This is Python's big surprise: it uses **indentation** (spaces) to group code — there are no `{}` braces. **(If you already code:** yes, whitespace is *meaningful* here — the indent IS the block.**)** Forgetting the colon, or getting the spacing wrong, is the most common early error. Line things up neatly and you're fine.

else — the other road



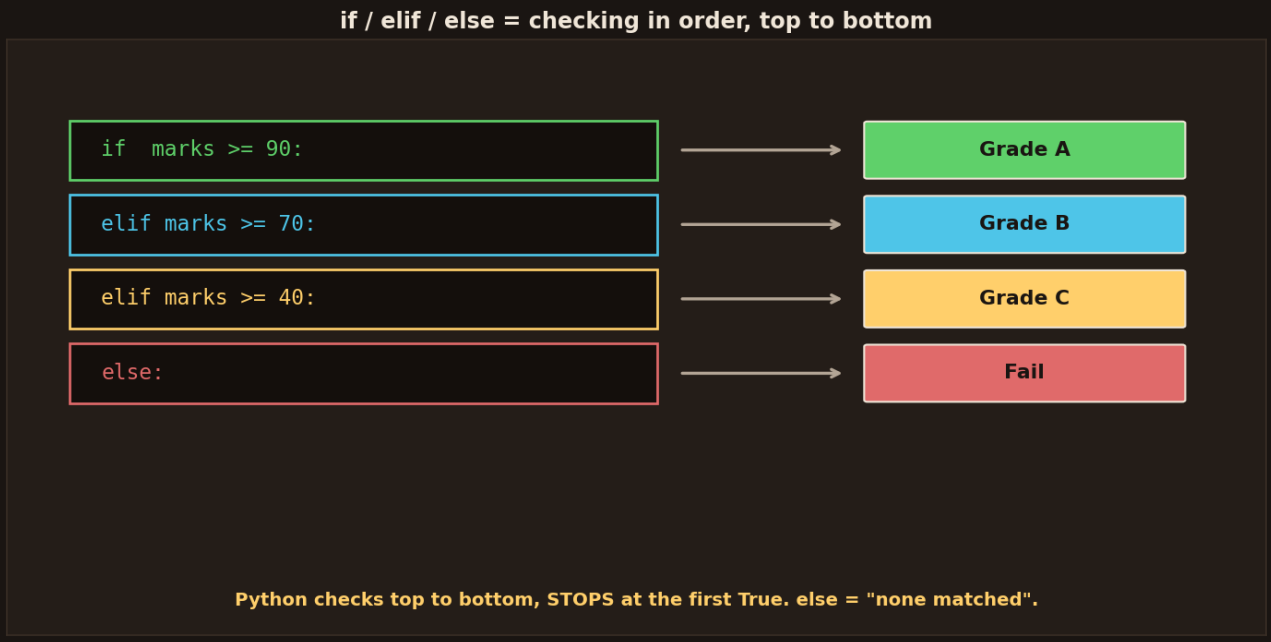
Indentation decides what belongs inside the if. The two indented lines run only when logged_in is True; the un-indented print("Done") always runs. Line up the spaces — 4 is standard.

Often you want to do one thing if true, and **something else** if not. That's `else`:

```
age = 15
if age >= 18:
    print("You may enter")
else:
    print("Sorry, too young")
# -> Sorry, too young
```

> `else` is the "otherwise" road — it runs **only when the `if` was False.** No condition after `else` (it's the catch-all), just a colon and an indented block. Now your program has two clear paths, and exactly one of them always runs.

elif — more than two paths



if/elif/else checks in order, top to bottom. if marks>=90 gives Grade A, elif marks>=70 gives Grade B, elif marks>=40 gives Grade C, else gives Fail. Python stops at the first True; else means none matched.

What if there are *several* possibilities — like grading marks A/B/C/Fail? Chain them with `elif` (short for "else if"):

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 70:
    print("Grade B")
elif marks >= 40:
    print("Grade C")
else:
    print("Fail")
# -> Grade B
```

> Read it as a ladder: check the first — if not, check the next — and so on. `elif` lets you handle as many cases as you like. The final `else` is the catch-all for "none of the above." One `if`, any number of `elif`s, an optional `else`.

It stops at the first match

With marks = 75: marks>=90 is False (skip), marks>=70 is True so it runs Grade B and STOPS. The marks>=40 and else are never checked. Result: Grade B. Order matters.

Here's the key rule that catches people: Python checks the conditions **top to bottom and stops at the FIRST one that's True.**

- `marks = 75`: `>= 90`? No. `>= 70`? **Yes** → print "Grade B" and **stop.**
- The `>= 40` check and the `else` are never even looked at.

> Because it stops at the first match, **order matters.** If you put `>= 40` first, then 75 would wrongly get "Grade C" — because 75 is also ≥ 40, and it'd stop there. Rule of thumb: put the **strictest / highest** condition first, work down to the loosest. Get the order wrong and the logic quietly breaks.

Recap — the 20-second version

- `if condition:` runs the indented block **only when the condition is True.**
- The line ends with a **colon** `:`; the block is **indented** (4 spaces) — no `{ }`.
- `else:` is the "otherwise" road — runs when the `if` was False.
- `elif` adds more paths; use as many as you need, with an optional `else`.
- Python **stops at the first True** — so put the strictest condition first.

> Next up — Video 8: Type Conversion & Errors. Remember input() always gives text? Next we learn to convert between types — int(), str(), float() — safely, and how to read the error messages Python shows when something goes wrong. It's the last piece of Part 1. See you tomorrow.